APPENDIX · TECHNICAL DEPTH

Knowhow

Twelve slides of the math, the tests, and the code patterns that back the headline numbers.

INDEX

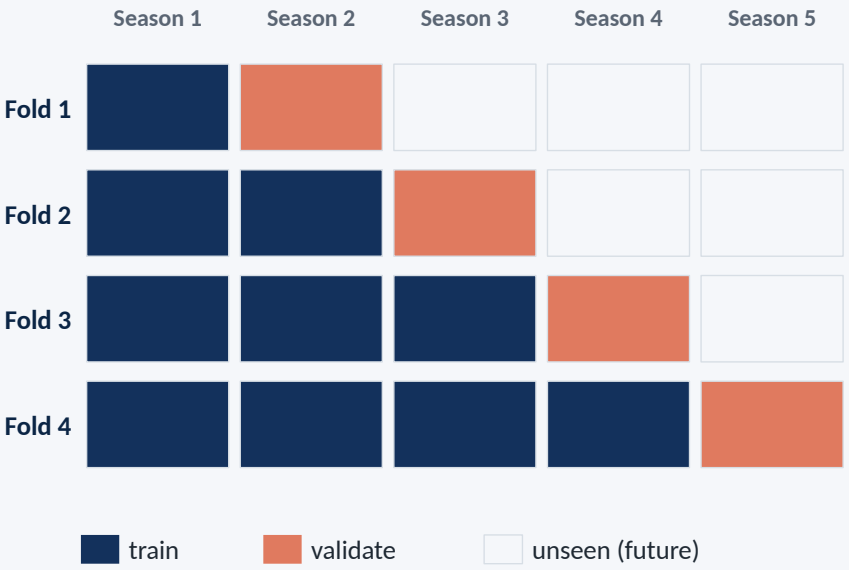
A1 Walk-forward CV A2 Log-loss vs accuracy A3 Brier decomposition
A4 Dixon-Coles math A5 Time-decay weighting A6 ClubElo update
A7 Vig removal + Kelly A8 NLP attribution A9 PCA + K-means clusters
A10 Team-name harmonisation A11 Anti-leakage unit test A12 Posit Cloud deploy

Walk-forward CV — never k-fold

Random k-fold leaks the future.
Folds drawn uniformly mix matches from 2020 with matches from 2010 — the model can use a 2020 home-form average to predict a 2010 outcome.

Walk-forward expanding window.
At fold k, train on seasons 1..k, validate on season k+1. Mirrors how the model would actually be used in production: every prediction uses only data available before kickoff.

Reported metric is the mean over folds; we also report per-season log-loss to show stability across the test windows.



Why log-loss is the headline metric

Accuracy is too coarse for 3-class probabilistic forecasting.
It throws away the model's confidence. A model that says (0.50, 0.20, 0.30) and one that says (0.95, 0.02, 0.03) are scored identically when the home team wins.

Log-loss is a proper scoring rule.
It is uniquely minimised when the predicted distribution equals the true distribution. Confidently wrong predictions are penalised asymptotically — that's why Always-Home gets log-loss 22.4.

We also report Brier (alternative proper rule) and AUC OvR (rank-based).

Multiclass log-loss

$$L(y, \hat{p}) = -(1/N)\, \Sigma_i\, \Sigma_c \;\; 1\{y_i = c\} \cdot \log \hat{p}_{i,c}$$

Predicted p̂(true class)	Per-row log-loss
0.95	0.05
0.50	0.69
0.20	1.61
0.05	3.00
0.001	6.91

Brier-score decomposition

Brier complements log-loss with a finite penalty for any error. Murphy (1973) decomposes a Brier score into three interpretable parts:

Reliability

Lower is better. Distance between predicted probability bins and observed frequencies. Calibration error in disguise.

Resolution

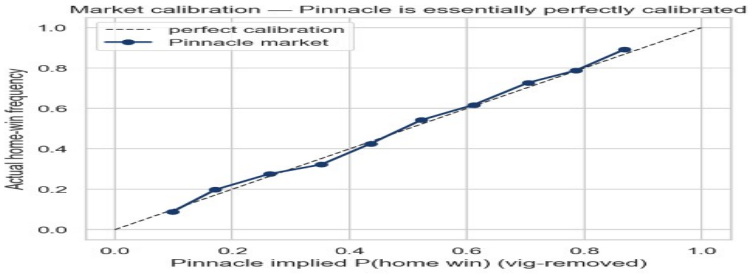
Higher is better. How much sharper the conditional distributions are than the marginal — i.e. is the model differentiating?

Uncertainty

Irreducible. Variance of the marginal class distribution. Football's intrinsic noise floor.

Murphy decomposition

$$Brier = Reliability - Resolution + Uncertainty$$



Pinnacle close: essentially on the diagonal — pure calibration.

Dixon-Coles bivariate Poisson

Per-team latent attack and defence.

```
α_t  =  attack strength of team t
β_t  =  defensive frailty (positive → leaks goals)
γ     =  global home-advantage offset
ρ     =  low-score correlation correction
```

Identifiability constraint: Σ_t α_t = 0.
Fitted by SLSQP-constrained MLE on a 0.0065/day exponentially decayed sample.

Marginalise the joint to recover H/D/A:

```
p_H = Σ_{x>y} P(x,y)  ·  p_D = Σ_x P(x,x)  ·  p_A = Σ_{x<y} P(x,y)
```

Match-level scoring rates

$$\lambda = exp(\alpha_h + \beta_a + \gamma)$$
$$\mu = exp(\alpha_a + \beta_h)$$

Joint pmf with low-score correction

$$P(X=x, Y=y) = \tau(x,y; \lambda,\mu,\rho) \cdot Pois(x;\lambda) \cdot Pois(y;\mu)$$

tau τ — fixes 0-0 / 1-1 underprediction

$$\tau(0,0)=1-\lambda\mu\rho \quad \tau(0,1)=1+\lambda\rho$$
$$\tau(1,0)=1+\mu\rho \quad \tau(1,1)=1-\rho$$

Time-decay weighting

Football is non-stationary.
Manager turnover, transfers, tactical evolution — a match from 2002 should not be weighted equally to one from last week.

Exponential decay weight in the likelihood:

Weight of a historical match i

$$w_i = exp(-\lambda \cdot days_old_i)$$

`λ = 0.0065 / day` gives a half-life of about 107 days — roughly 14 matches.

Tuned by walk-forward log-loss; flat (λ=0) loses to decayed by ~0.01.

WEIGHT-VS-AGE CURVE (placeholder)

figures/decay_weight_curve.png

Match age	Weight
30 days	0.82
107 days (half-life)	0.50
1 year	0.094
3 years	0.001

ClubElo update mechanics

Two strength signals.
1. ClubElo (clubelo.com) — third-party, daily, since 2000.
2. Our own Elo, recomputed from football-data.co.uk results.

Update rule (Elo / FIFA-Maher style):

Expected score & rating update

$$E_h = 1 / (1 + 10^{((R_a - R_h - HFA)/400)})$$

$$R_h \leftarrow R_h + K \cdot (S_h - E_h)$$

K = 20 (typical), **HFA = 65** home-advantage points (pre-Covid).

WORKED EXAMPLE

Liverpool 1 – 1 Tottenham

```
Pre-match:     R_h = 1920   R_a = 1820
HFA-adj diff: (1820 - 1920 - 65) / 400 =
-0.4125
E_h = 1 / (1 + 10^-0.4125) = 0.721
S_h = 0.5  (a draw)

Update:        R_h ← 1920 + 20·(0.5 - 0.721)
             = 1915.6  (drops 4.4)
               R_a ← 1820 + 4.4 = 1824.4
```

Vig removal and Kelly-fractional sizing

Vig removal — Pinnacle close

$$p_raw,c = 1 / odds_c$$
$$overround = \Sigma_c\ p_raw,c \quad (\approx 1.05)$$
$$p_novig,c = p_raw,c\ /\ overround$$

Kelly — fraction of bankroll to stake

$$f^* = (b \cdot p - q) / b$$
$$where\ \ b = decimal_odds - 1,\ \ q = 1 - p$$

Pinnacle's overround is ~5%.
Renormalising p_H + p_D + p_A → 1 strips the bookmaker margin and gives a usable probability estimate. Pinnacle is the closest-to-fair book; other books need their bigger overround removed before comparison.

f* maximises expected log-bankroll.
We cap f* at 5% of bankroll per bet ("fractional Kelly"). Full Kelly's variance is brutal — the cap halves drawdown for ~80% of expected growth.

`Place only when edge = p · odds − 1 > 0.`

Size by the formula above; track bankroll trajectory.

spaCy + VADER per-team sentence attribution

Algorithm sketch.

1. Strip BBC/Guardian boilerplate via BeautifulSoup; concat <article> text.
2. Sentence-split on the standard regex `(?<=[.!?])\s+`.
3. For each sentence, attribute to the team whose canonical name appears uniquely in that sentence (skip mixed-mention sentences).
4. Score each per-team sentence with VADER; aggregate by mean.
5. Run spaCy en_core_web_sm NER once on the full doc; count PERSON entities.
6. Apply regex keyword classifier (red card / penalty / VAR / injury / controversy).

Output ~12 derived columns per match.

src/nlp/parse_match_report.py

```
def per_team_sentiment(text, h_t, a_t):
    sia = _sentiment()
    sents = re.split(
        r"(?<=[.!?])\s+", text)
    hl, al = h_t.lower(), a_t.lower()
    h, a = [], []
    for s in sents:
        sl = s.lower()
        c = sia.polarity_scores(s)[
            "compound"]
        if hl in sl and al not in sl:
            h.append(c)
        elif al in sl and hl not in sl:
            a.append(c)
    return mean(h), mean(a)
```

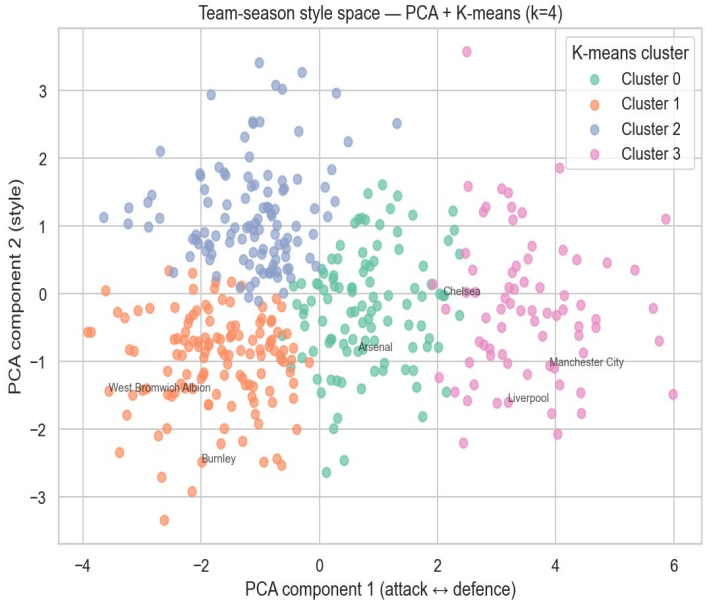
PCA + K-means team-style clusters

Inputs. Per-team-per-season aggregate stats: shots/match, possession, passes-completed, xG-for, xG-against, set-piece share, fouls.

Pipeline. StandardScaler → PCA(n=2) → KMeans(k=4, random_state=20260426).

Why k=4. Elbow at k=4 on the within-cluster sum of squares; silhouette also peaks around k=4. The clusters correspond to interpretable archetypes:

k	Archetype	Typical clubs
0	Possession-dominant	Man City, Liverpool, Chelsea
1	Counter-attacking	Leicester '15-16, Spurs '21
2	Defensive low-block	Burnley, Stoke '12-15
3	Mid-table mix	West Ham, Everton, Newcastle



PCA-2 with K-means (k=4) — see clusters identified on facing page.

Team-name harmonisation — fail fast on unknowns

The silent-mis-join trap.
football-data.co.uk uses `"Man United"`, FBref uses `"Manchester Utd"`, BBC uses `"Manchester United"`, Wikipedia uses `"Manchester United F.C."`. A naive merge silently drops ~40% of rows and corrupts every feature downstream.

Our pattern.

1. Define a canonical name dict (Wikipedia article title minus F.C.).
2. Every variant is normalised case + suffix-stripped.
3. `canonical()` raises `UnknownTeamError` on miss — caller fails loudly.
4. CI runs the canonicaliser over every scraped row on every push.

src/team_harmonize.py

```
class UnknownTeamError(KeyError):
    pass

_TEAM_VARIANTS = {
    "Manchester United": (
        "Man United", "Man Utd",
        "Manchester Utd",
    ),
    "Tottenham Hotspur": (
        "Tottenham", "Spurs",
    ),
    ...
}

def canonical(name: str) -> str:
    key = _normalize(name)
    if key not in _MAP:
        raise UnknownTeamError(name)
    return _MAP[key]
```

Anti-leakage invariant — what the test checks

The invariant.
For every row m in the processed feature matrix and every rolling feature column f, the value of f[m] depends only on rows whose match_date is strictly less than m.match_date.

What would break.
Forgetting a `.shift(1)` on a rolling-window aggregate would let the current match's own goals leak into its own form-feature.

What the test does.
Build features twice — once with the full dataset, once with the dataset truncated to seasons ≤ 2018-19. For every match in 2018-19, the feature values must match exactly across the two builds. *If they differ, future data leaked.*

tests.test_no_temporal_leakage

```
def test_no_temporal_leakage():
    raw = load_interim_matches()
    full = build_processed_matches(raw)

    cutoff = pd.Timestamp("2019-08-01")
    early = raw[raw.match_date < cutoff]
    truncd = build_processed_matches(early)

    cols = [c for c in full.columns
            if c.startswith(("home_form",
                             "away_form"))]

    full_pre = full[full.match_date < cutoff]
    pdt.assert_frame_equal(
        full_pre[cols].reset_index(drop=True),
        truncd[cols].reset_index(drop=True),
    )
```

Posit Connect Cloud deployment workflow

1	**Pre-bake artefacts**	Run scrape → train → evaluate locally; commit results/leaderboard.csv plus the .joblib model files into the repo.
2	**Pin the environment**	requirements.txt with exact versions; rsconnect-python verifies and replays the env in the cloud build.
3	**rsconnect deploy**	`rsconnect deploy shiny app.py --server connect.posit.cloud --api-key $RSC_API_KEY`
4	**Smoke-test in cloud**	First request loads models; warm-time ~6s; subsequent requests <300ms because Shiny keeps state.
5	**Pin the URL**	Paste the deployed URL into the oral deck slide 11 and the README before submission.